AZURE AI FOUNDRY • TECHNICAL ARCHITECTURE & IMPLEMENTATION BLUEPRINT

Deterministic Confidence & Accuracy for RAG-Based PDF JSON Extraction

A zero-hallucination mathematical framework for attribute extraction using database definitions, indexed PDF chunks, token log-probabilities, and automated benchmark verification.

1. Architectural Context & Problem Statement

In enterprise document processing pipelines built on **Azure AI Foundry**, source PDFs are chunked and indexed into Azure AI Search while target attribute names and business definitions are maintained dynamically in a relational or NoSQL database. At runtime, relevant PDF chunks are retrieved via Retrieval-Augmented Generation (RAG) and paired with database attribute definitions so the Large Language Model (LLM) extracts a structured key-value JSON output. Engineering teams require a mathematically logical, official mechanism to score extraction reliability that guarantees **identical, deterministic results across N repeated runs** with **zero hallucinated values**.

Critical Anti-Pattern: Why Prompted Self-Confidence Fails — Non-Deterministic

Instructing an LLM inside a prompt to *"rate your confidence from 1 to 100 for each extracted key"* produces arbitrary, hallucinated numbers. Autoregressive models do not introspect their internal calibration via text generation; an LLM will routinely assign 95% confidence to a fabricated value. Reliable scoring requires separating **Runtime Confidence** from **Benchmark Accuracy**.

A. Runtime Confidence Score — Per New PDF (No Ground Truth)

Calculated dynamically in production for every incoming PDF when no verified answer key exists. Combines intrinsic model token generation probability (**logprobs**) with a deterministic Python substring verification gate against retrieved PDF chunks.

B. Benchmark Extraction Accuracy — Across N Runs (Golden Dataset)

Calculated during CI/CD testing and validation against a verified "Golden Dataset" (ground-truth JSONL). Uses deterministic code-based evaluators in the **Azure AI Evaluation SDK** to compute Exact Match Accuracy, Precision, Recall, and F_1.

2. The Three Official Approaches in Azure AI Foundry

Foundry Approach	Best Used When	Official Technical Mechanism	Determinism
1. Azure AI Content Understanding	Extracting attributes directly from PDFs using schema definitions without custom chunk management.	Built-in `estimateFieldSourceAndConfidence = true` property inside the Document Analyzer schema.	**100% Deterministic** (Native token score + bounding boxes)
2. Custom RAG + Azure OpenAI (logprobs)	Extracting JSON from Azure AI Search PDF chunks using dynamic DB attribute definitions.	OpenAI `logprobs=True` + Strict JSON Schema + Deterministic Python Substring Quote Gate.	**100% Deterministic** (Token math + exact code verification)
3. Azure AI Evaluation SDK (`azure-ai-evaluation`)	Benchmarking pipeline extraction accuracy across *N* test runs against ground truth.	Custom Code-Based Evaluator classes + `evaluate()` pipeline runner on Golden JSONL files.	**100% Deterministic** (Avoids LLM-as-a-judge variance)

3. Calculating Runtime Confidence Score (No Ground Truth)

When extracting attributes from retrieved PDF chunks in production, you can compute an official confidence score per attribute and for the overall JSON using either native Content Understanding analyzers or custom RAG token log-probabilities gated by verbatim quote verification.

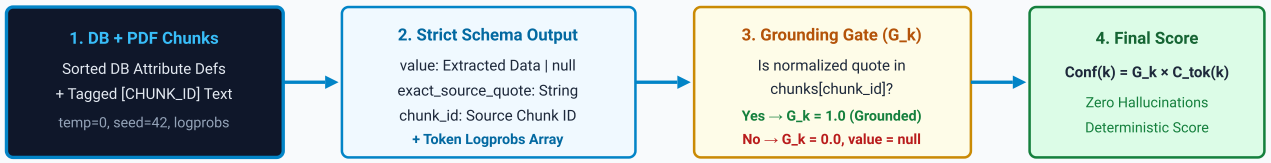


Figure 1: End-to-end runtime confidence scoring flow combining OpenAI token log-probabilities with deterministic Python substring verification.

Option A: Native Confidence via Azure AI Content Understanding

If your pipeline uses an **Azure AI Content Understanding** Document Analyzer (GA API `2025-11-01` or `2026-06-01-preview`), inject your database attribute definitions directly into each field's `description` and enable native confidence estimation:

```
{ "fieldSchema": { "estimateFieldSourceAndConfidence": true, "fields": {
"ContractEffectiveDate": { "type": "date", "method": "extract", "description": "DB
Definition: The legal date on which the agreement takes effect.",
"estimateSourceAndConfidence": true } } } }
```

Option B: Custom RAG Pipeline Mathematical Confidence Model (logprobs + Quote Gate)

1. **Enforce Evidence in Strict JSON Schema:** Dynamically build a Strict Structured Output schema from your DB definitions requiring three fields per attribute: `value`, `exact_source_quote` (verbatim substring from the chunk), and `chunk_id`.
2. **Enable Token Log-Probabilities:** Pass `logprobs=True` and `top_logprobs=1` in the Azure OpenAI completion call. The API returns the natural log probability `ln(p)` for every generated token.
3. **Compute Linear Token Probability $C_{token}(k)$:** For the *T* tokens composing the extracted value of attribute *k*, compute the geometric mean probability by exponentiating the arithmetic mean of token log-probabilities.
4. **Apply the Deterministic Grounding Gate (G_k):** In Python, normalize whitespace and case and verify whether `exact_source_quote` exists as an exact substring inside the retrieved PDF chunk text (`chunks[chunk_id]`). If verified, set $G_k = 1.0$. If the quote does not exist in the chunk, a hallucination occurred: set $G_k = 0.0$ and force `value = null`.

1. Token Probability for Attribute k: $C_{token}(k) = \exp\left((1 / T) \times \sum_{i=1..T} \text{logprob}_i \right)$

2. Gated Attribute Confidence: $\text{Confidence}(k) = G_k \times C_{token}(k)$ where $G_k \in \{0.0, 1.0\}$

3. Overall Document JSON Confidence: $\text{Overall Confidence} = (1 / M) \times \sum_{k=1..M} \text{Confidence}(k)$

4. Calculating True Extraction Accuracy (Benchmarking via SDK)

To measure true extraction accuracy across *N* test runs, evaluate your pipeline against a verified **Golden Dataset** (a JSONL file pairing test PDF chunks with expected ground-truth JSON outputs) using the official `azure-ai-evaluation` SDK. Avoid LLM-as-a-judge evaluators when strict determinism is required; instead, register a **Custom Code-Based Evaluator** that classifies every attribute *k* into four deterministic outcomes:

- **True Positive (TP):** Ground truth has a value, extracted JSON has a value, and normalized values match exactly.
- **True Negative (TN):** Ground truth is `null` (attribute absent in PDF), and extracted JSON correctly outputs `null`.
- **False Positive (FP — Hallucination / Mismatch):** Ground truth is `null` but model extracted a value, OR extracted value mismatches ground truth.
- **False Negative (FN — Missed Extraction):** Ground truth has a value, but the model returned `null`.

$$\text{Exact Match Accuracy} = (TP + TN) / (TP + TN + FP + FN) \quad | \quad \text{Precision} = TP / (TP + FP)$$
$$\text{Recall} = TP / (TP + FN) \quad | \quad F_1\ \text{Score} = (2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

5. Engineering Complexity & Pipeline Integration Delta

When your PDF chunking, Azure AI Search index submission, database attribute lookup, and Foundry model deployment are already working, adding deterministic confidence scoring requires **2 to 4 hours of backend engineering** and **zero changes to your indexing or database infrastructure**:

Existing Pipeline Stage	Modification	Effort	Required Engineering Action
1. PDF Ingestion & Indexing	None (0%)	0 mins	Keep existing PDF chunking and Azure AI Search index submission unchanged.
2. DB Attribute Definitions	None (0%)	0 mins	Keep existing database schema and attribute definition queries unchanged.
3. Foundry Model Deployment	None (0%)	0 mins	No fine-tuning or model replacement needed; pin model version snapshot.
4. Chunk Context Formatting	Minor Tweak	~15 mins	Prefix retrieved chunks in the prompt with `[CHUNK_ID: id]` and sort DB keys alphabetically.
5. OpenAI Extraction Call	Config Update	~20 mins	Add `temperature=0.0, seed=42, logprobs=True`; expand schema per key to include `exact_source_quote` and `chunk_id`.
6. Post-Processing Scorer	New Wrapper	~2–3 hrs	Add a ~50-line Python function after the LLM call to compute `exp(mean(logprobs))` and enforce the substring quote gate.

Choosing Your Implementation Path by Granularity

Path 1: Whole-JSON + Gate ~1.5 Hrs

Average token `logprobs` across the entire JSON response and run the per-attribute Python substring quote check. Zero token-slicing complexity; guarantees zero

Path 2: Per-Key Slicing ~3.5 Hrs

Slice the flat OpenAI `logprobs` token array by JSON key character offsets to compute a distinct token probability for each individual attribute alongside the quote gate.

Path 3: Native Analyzer ~15 Mins

If using Azure AI Content Understanding instead of custom RAG chunks, enable `estimateFieldSourceAndConfidence: true` in the analyzer schema for built-in scoring.

6. Reference Python Implementation (Scorer & SDK Evaluator)

```
import math def normalize_text(text: str) -> str: """Deterministic whitespace and lowercase normalization without regex overhead.""" return " ".join(str(text).strip().lower().split()) if text else "" def verify_and_score_extraction(extracted_json: dict, chunks_by_id: dict, logprobs_content: list) -> dict: """Computes token confidence from logprobs and enforces the deterministic substring quote gate.""" token_probs = [math.exp(t.logprob) for t in logprobs_content] avg_token_conf = sum(token_probs) / len(token_probs) if token_probs else 0.0 verified_output, field_scores = {}, [] for attr, payload in sorted(extracted_json.items()): val, quote, cid = payload.get("value"), payload.get("exact_source_quote"), payload.get("chunk_id") if val is None: verified_output[attr] = {"value": None, "confidence": round(avg_token_conf, 4), "is_grounded": True} continue # DETERMINISTIC ZERO-HALLUCINATION GATE: Verify verbatim quote exists inside cited PDF chunk chunk_text = chunks_by_id.get(cid, "") is_grounded = bool(quote) and (normalize_text(quote) in normalize_text(chunk_text)) if not is_grounded: verified_output[attr] = {"value": None, "confidence": 0.0, "is_grounded": False, "rejected": val} field_scores.append(0.0) else: verified_output[attr] = {"value": val, "confidence": round(avg_token_conf, 4), "is_grounded": True} field_scores.append(avg_token_conf) overall = sum(field_scores) / len(field_scores) if field_scores else 1.0 return {"attributes": verified_output, "overall_confidence": round(overall, 4)} class DeterministicJsonExtractionEvaluator: """Custom code-based evaluator for azure-ai-evaluation guaranteeing identical scores across N runs.""" def __call__(self, *, response: dict, ground_truth: dict, **kwargs) -> dict: tp = tn = fp = fn = 0 for key, expected in sorted(ground_truth.items()): raw_actual = response.get(key, {}).get("value") if isinstance(response.get(key), dict) else response.get(key) exp_norm = normalize_text(expected) if expected is not None else None act_norm = normalize_text(raw_actual) if raw_actual is not None else None if exp_norm is None and act_norm is None: tn += 1 elif exp_norm is not None and act_norm is None: fn += 1 elif exp_norm is None and act_norm is not None: fp += 1 elif exp_norm == act_norm: tp += 1 else: fp += 1 total = tp + tn + fp + fn acc = (tp + tn) / total if total else 0.0 prec = tp / (tp + fp) if (tp + fp) else 0.0 rec = tp / (tp + fn) if (tp + fn) else 0.0 f1 = (2 * prec * rec) / (prec + rec) if (prec + rec) else 0.0 return {"exact_match_accuracy": round(acc, 4), "precision": round(prec, 4), "recall": round(rec, 4), "f1_score": round(f1, 4)}
```

7. Five-Point Checklist to Guarantee Determinism Across N Runs

1. **Lock Azure OpenAI Generation Hyperparameters:** Always pass `temperature=0.0`, `top_p=1.0`, `seed=42` (or a fixed integer), and `logprobs=True`. Pin an explicit model version snapshot (such as `gpt-4o-2024-08-06`) and log `response.system_fingerprint` to verify backend hardware consistency.
2. **Sort Database Attributes Alphabetically:** Relational and NoSQL queries do not guarantee row order. Always sort attribute keys alphabetically (`sorted(db_attributes.items())`) before constructing the prompt and JSON schema so the autoregressive token sequence never shifts.
3. **Enforce Deterministic Tie-Breaking on Retrieved Chunks:** Distributed vector search indexes can return tied similarity scores in varying order across runs. Sort retrieved chunks deterministically in Python before prompt injection: `chunks.sort(key=lambda c: (-round(c["score"], 4), c["chunk_id"]))`.
4. **Explicitly Allow `null` in Strict JSON Schemas:** Forced hallucinations happen when a schema mandates `"type": "string"` for an attribute absent from the PDF. Define every attribute value type as `["string", "number", "boolean", "null"]` and instruct the model to output `null` when unverified.
5. **Gate Every Extracted Value by Substring Verification:** Never pass an LLM-extracted value downstream unless its accompanying `exact_source_quote` passes the deterministic normalized substring check against the retrieved chunk text.